Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Описание архитектуры

Платформа, компоненты, решения и план развития · версия 0.3.0

28 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Общие сведения 4

2.1 Назначение и класс системы 4

2.2 Архитектурные принципы 4

2.3 Показатели кодовой базы 5

3 Контекст системы 6

4 Контейнеры и развёртывание 7

4.1 Состав контейнеров 7

4.2 Поставка и окружения 8

4.3 Масштабирование 8

5 Серверная часть 10

5.1 Слои и компоненты 10

5.2 Ключевые сервисы 11

5.3 Обработка запроса 12

6 Клиентская часть 13

7 Модель данных 15

7.1 Организация 15

7.2 Отпуска и режим работы 16

7.3 Служебные данные 16

7.4 Жизненный цикл заявки 17

8 Ключевые сценарии 18

8.1 Вход и продление сессии 18

8.2 Подача заявки 19

8.3 Согласование и отклонение 20

8.4 Построение графика 21

9 Безопасность 22

9.1 Аутентификация и сессии 22

9.2 Защита запросов 22

9.3 Модель авторизации 23

9.4 Журналирование и аудит 23

10 Программный интерфейс 24

11 Наблюдаемость и эксплуатация 25

12 Технологический стек 26

13 Ключевые решения 27

14 План развития 28

1 О документе

Документ описывает архитектуру «Планировщика отпусков» версии 0.3.0: назначение и границы системы, состав компонентов и их взаимодействие, модель данных, программный интерфейс, механизмы безопасности, развёртывание, эксплуатацию и план развития. Документ предназначен для архитекторов, разработчиков, инженеров сопровождения и службы информационной безопасности.

СВЯЗАННЫЙ ДОКУМЕНТ	СОДЕРЖАНИЕ
0 — Ревизия функциональности	Что было в прототипе и что стало целевым составом
2 — Спецификация API v1	Протокол, каталог операций и ошибок, матрица прав
3 — План развития и бэклог	Версии, эпики и истории с критериями приёмки
4 — Модель данных	Сущности, словарь данных, инварианты, миграции
7 — Стенды, CI/CD и выпуск	Окружения, конвейеры, порядок выпуска
11 — SBOM и лицензии	Состав сторонних компонентов для реестра отечественного ПО
Руководство по развёртыванию	Установка, обновление, резервное копирование

Источники истины, на которые опирается документ: контракт `contracts/openapi-vacation-api-v1.yaml`, схема `server/prisma/schema.prisma` и исходный код ветки `v0.3.0`. Показатели кодовой базы ниже сформированы автоматически (`tools/docs/gen_tables.py`), а соответствие контракта и реализации проверяется тестом в CI.

2 Общие сведения

2.1 Назначение и класс системы

«Планировщик отпусков» — корпоративное веб-приложение для планирования и согласования ежегодных отпусков. Оно ведёт график отпусков всей организации, заявки сотрудников с согласованием руководителем, балансы дней по нормам Трудового кодекса, режим работы офис/удалёнка, аналитику пересечений внутри отделов и неизменяемый журнал всех изменений.

По архитектурному типу это **клиент-серверное веб-приложение**: одностраничный клиент (SPA) и REST API, устанавливаемые на серверы организации (on-premise) в контейнерах Docker. Целевой масштаб — от 1000 до 5000 пользователей.

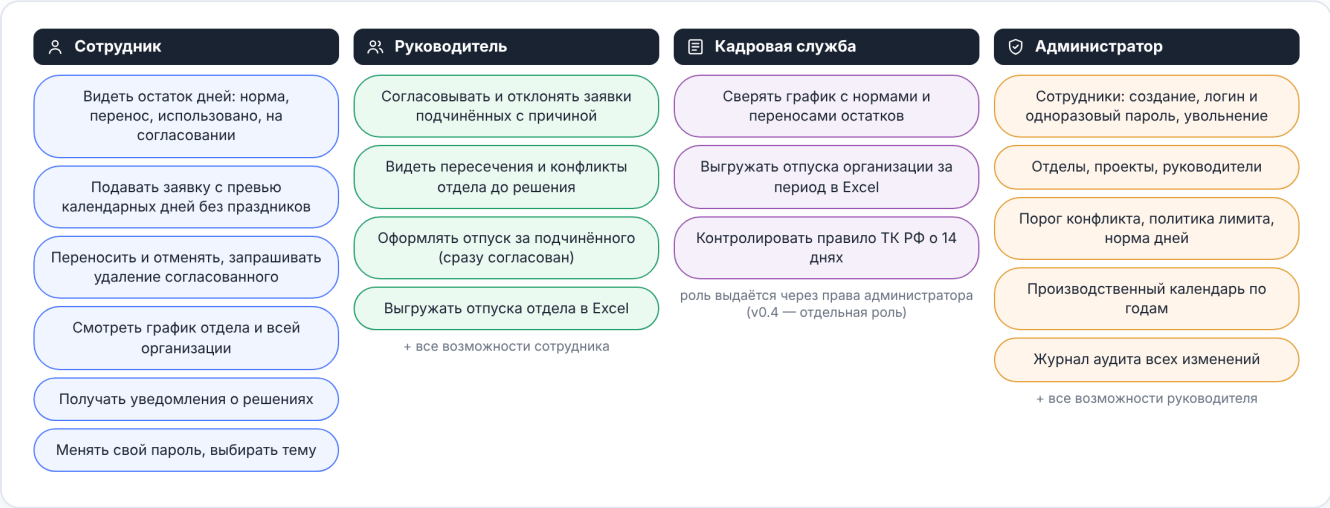


Рис. 1. Роли и их возможности

2.2 Архитектурные принципы

1. **Сервер решает, клиент показывает.** Правила (конечный автомат заявки, балансы, лимиты, права, конфликты) реализованы только в серверных сервисах. Проверки в клиенте — подсказки для удобства, а не защита.
2. **Контракт прежде кода.** REST API v1 описан в `contracts/` и меняется через ревью владельца; тест сверки в CI не даёт реализации и контракту разойтись.
3. **Одна операция — одна транзакция.** Изменение данных, запись в историю заявки, журнал аудита и уведомления фиксируются в одной транзакции PostgreSQL.
4. **Чистые вычисления отдельно от ввода-вывода.** Баланс, интервалы и аналитика пересечений — модули без БД и без «текущего времени»: их проверяют модульные тесты, а дата расчёта передаётся аргументом.
5. **Никаких временных решений.** Зрелый стек с первой версии: PostgreSQL, TypeScript strict, контейнеры, миграции, CI с e2e-тестами.

6. **Зависимости — под контролем.** Каждая новая библиотека обосновывается в `docs/decisions.md` и проходит лицензионный гейт SBOM: copyleft и неизвестные лицензии блокируют слияние.
7. **Секреты — только в окружении.** В репозитории нет паролей и ключей; сервер не стартует со слабым `JWT_SECRET`.

2.3 Показатели кодовой базы

ПОКАЗАТЕЛЬ	ЗНАЧЕНИЕ
Сервер: файлов TypeScript / строк	43 / 3 249
Сервер: контроллеров / операций API	12 / 47
Модель данных: таблиц / миграций	14 / 1
Клиент: файлов TS и Vue / строк	37 / 9 059
Клиент: экранов	10
Тесты сервера: файлов / сценариев	7 / 54
Тесты клиента (модульные и e2e): файлов / сценариев	4 / 24

3 Контекст системы

Пользователи работают с системой через браузер по HTTPS. Внешние системы организации подключаются по мере развития (пунктир — плановые интеграции версии 0.4).

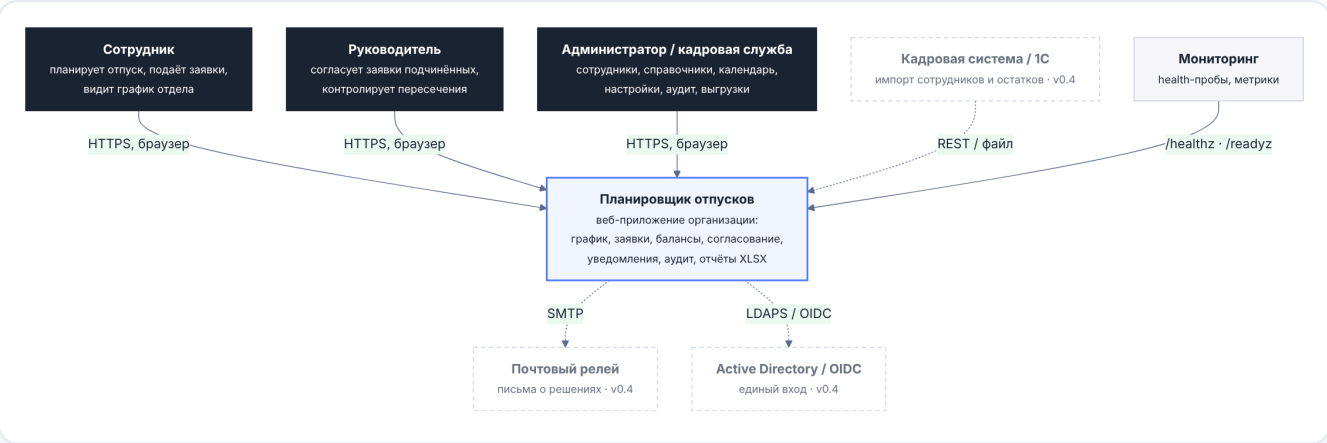


Рис. 2. Диаграмма контекста (уровень 1 C4)

УЧАСТНИК	ВЗАИМОДЕЙСТВИЕ
Сотрудник	Планирует отпуск, подаёт и переносит заявки, смотрит график отдела и свой график работы
Руководитель	Согласует заявки подчинённых, видит пересечения и аналитику отдела, ведёт режим работы команды
Администратор и кадровая служба	Сотрудники, справочники, производственный календарь, правила организации, журнал аудита, выгрузки
Мониторинг	Опрашивает `/healthz` (процесс жив) и `/readyz` (доступна БД)
Почтовый релей, кадровая система, AD/OIDC	Плановые интеграции: письма о решениях, импорт сотрудников и остатков, единый вход

4 Контейнеры и развёртывание

4.1 Состав контейнеров

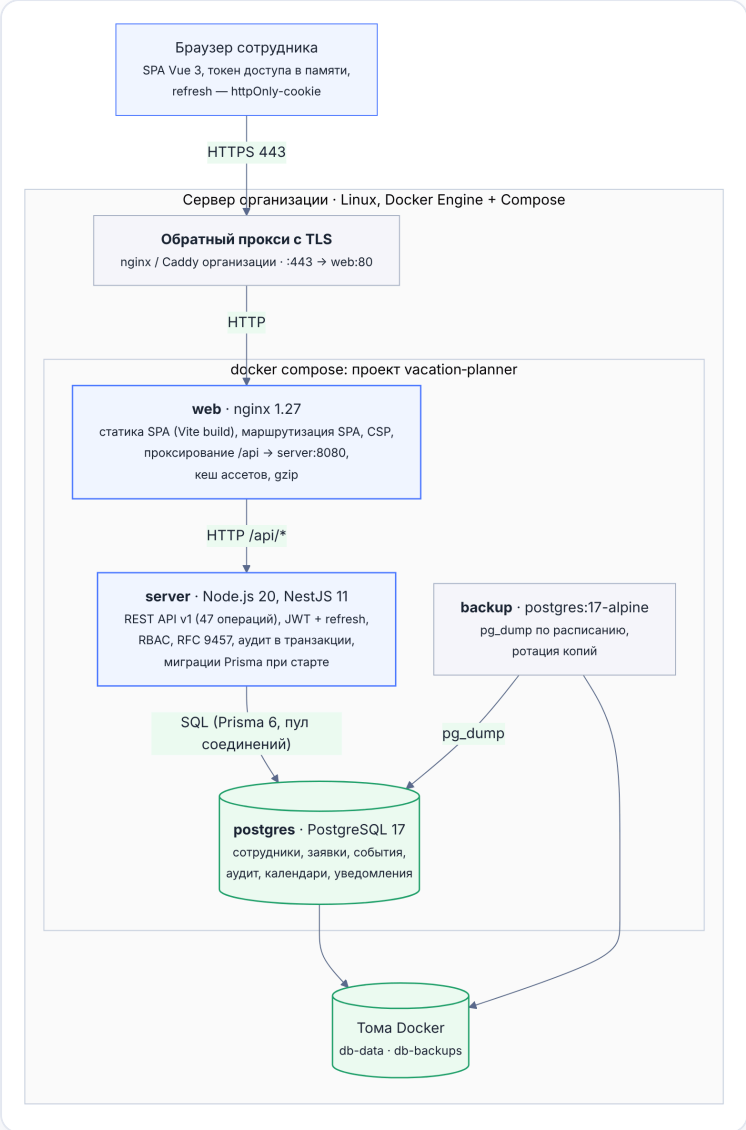


Рис. 3. Диаграмма контейнеров (уровень 2 C4)

КОНТЕЙНЕР	ОБРАЗ	НАЗНАЧЕНИЕ
`web`	nginx 1.27 + статическая сборка Vite	Отдаёт SPA с CSP и заголовками безопасности, кеширует хешированные ассеты, проксирует `/api` на `server:8080` с `X-Forwarded-For`

КОНТЕЙНЕР	ОБРАЗ	НАЗНАЧЕНИЕ
`server`	Node.js 20, NestJS 11	REST API v1, бизнес-логика, миграции Prisma при старте контейнера
`postgres`	PostgreSQL 17	Все данные; том `db-data`
`backup`	postgres:17-alpine	`pg_dump` по расписанию (по умолчанию раз в сутки), ротация копий 14 дней; том `db-backups`

Перед стендом ставится обратный прокси организации с TLS (nginx или Caddy). Состояние сессий в сервере не хранится: access-токен — JWT, refresh-токены — в таблице PostgreSQL, поэтому сервер масштабируется горизонтально без «липких» сессий.

4.2 Поставка и окружения

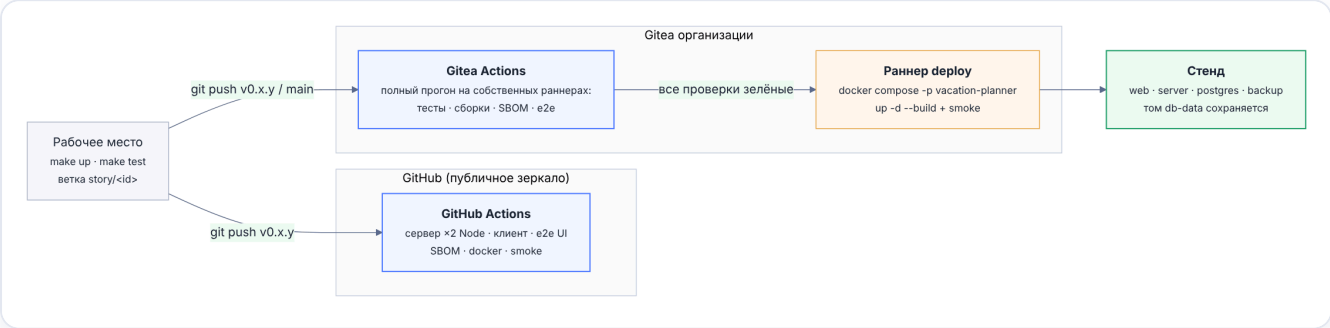


Рис. 4. Поставка: от рабочего места до стенда

Код ведётся в двух репозиториях-зеркалах: GitHub (публичный) и Gitea организации. Оба запускают одинаковый набор проверок; в Gitea после зелёных проверок раннер `deploy` разворачивает стенд (`docker compose -p vacation-planner up -d --build`) и выполняет дымовые пробы. Подробно — документ 7.

4.3 Масштабирование

НАГРУЗКА	РЕШЕНИЕ
До 1000 пользователей	Один сервер: 2 vCPU, 4 ГБ ОЗУ, PostgreSQL в том же compose

НАГРУЗКА	РЕШЕНИЕ
До 5000 пользователей	Вынести PostgreSQL на отдельный узел; 2 экземпляра `server` за балансировщиком; размер пула Prisma задаётся `connection_limit` в `DATABASE_URL`; счётчики лимита входа — в общий стор (v0.5)
Пики начала дня (×5)	Тяжёлые чтения (`/schedule`, аналитика) ограничены периодом ≤ 370–400 дней и фильтром по отделу; клиент кэширует праздники на сессию

5 Серверная часть

5.1 Слои и компоненты

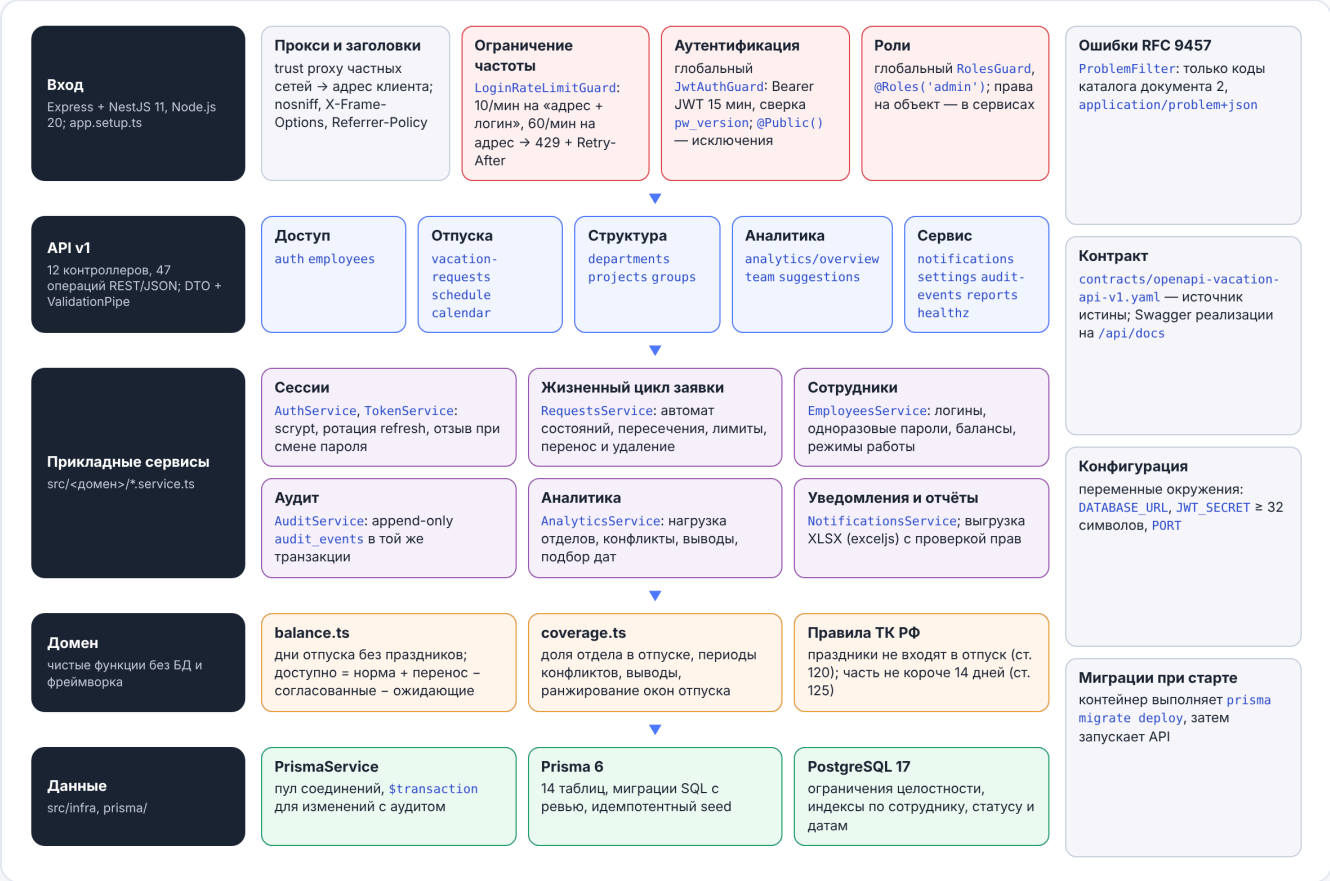


Рис. 5. Слои сервера (уровень 3 C4)

Сервер — модульный монолит NestJS: каждый домен в каталоге `server/src/<домен>/` содержит тонкий контроллер (DTO-валидация, вызов сервиса) и сервис с логикой.

МОДУЛЬ	ОТВЕТСТВЕННОСТЬ
`auth`	Вход, выход, обновление и смена пароля; `TokenService` (JWT HS256), глобальные guards `JwtAuthGuard` и `RolesGuard`, ограничение частоты входа
`app.setup.ts`	Общая настройка HTTP: trust proxy, cookie, заголовки, валидация — одна для продакшена и тестов
`employees`	Сотрудники, логины и одноразовые пароли, балансы, режим работы (шаблон и исключения)

МОДУЛЬ	ОТВЕТСТВЕННОСТЬ
`requests`	Заявки: конечный автомат, пересечения, лимиты, перенос и удаление; `balance.ts` — чистые функции баланса
`schedule`	График отпусков за период с конфликтами отделов
`analytics`	Нагрузка отделов, периоды конфликтов, выводы руководителю, подбор дат; `coverage.ts` — чистые функции
`directories`	Отделы, проекты, личные группы
`calendar`	Производственный календарь по годам
`settings`	Правила организации: порог конфликта, политика лимита, норма дней
`notifications`	Уведомления пользователей
`audit`	Неизменяемый журнал; запись только внутри транзакции изменения
`reports`	Выгрузка отпусков в XLSX
`health`	Пробы `/healthz` и `/readyz`
`infra`	`PrismaService`, `ProblemException` и фильтр RFC 9457

5.2 Ключевые сервисы

RequestsService реализует жизненный цикл заявки. Создание проверяет права (за себя, за подчинённого, администратор), даты (формат, порядок, не в прошлом), пересечения с собственными заявками и лимит дней. Заявка, оформленная руководителем за подчинённого, сразу согласована. Перенос согласованного отпуска переводит заявку на согласование с сохранением прежних дат в `change_from_*`; отклонение переноса возвращает прежние даты. Запрос удаления помечает заявку `pending_deletion`; согласование удаляет её, а событие остаётся в истории и аудите.

Баланс считается по правилам ТК РФ: праздничные нерабочие дни внутри отпуска не входят в число дней отпуска (ст. 120), одна из частей должна быть не короче 14 календарных дней (ст. 125). Доступно для планирования = норма + перенос − согласованные дни − дни на согласовании. При жёсткой политике лимита сервер отклоняет заявку сверх доступного (`409 limit-exceeded`).

AnalyticsService строит аналитику отдела на чистых функциях `coverage.ts`:

- доля отдела в отпуске по месяцам (только рабочие дни; отдельно — с учётом заявок на согласовании);
- периоды конфликтов, когда в отпуске не меньше порога отдела (выходные не разрывают период);
- выводы руководителю: ближайшие конфликты, пиковый и самый спокойный месяц, нераспределённые дни, сотрудники без части в 14 дней, риски малочисленного отдела;
- **подбор дат**: для заданной длины отпуска перебираются окна периода поиска; окно продлевается на праздники, окна с превышением порога и пересечением с собственными отпусками исключаются, остальные ранжируются по нагрузке на отдел, бонусу за праздники внутри и старту в понедельник; выдаётся до пяти непересекающихся вариантов с объяснением.

5.3 Обработка запроса

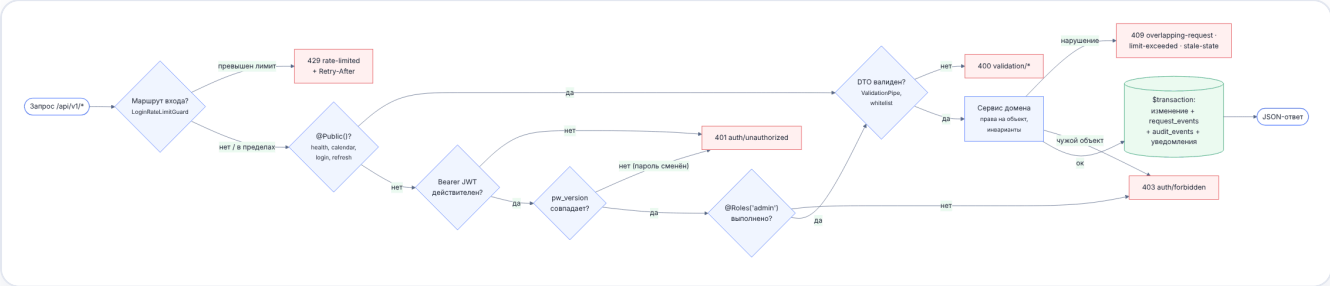


Рис. 6. Маршрут запроса через защиту и валидацию

Каждый запрос `/api/v1/*` проходит ограничение частоты (для входа), глобальную аутентификацию (кроме маршрутов `@Public()`), сверку версии пароля в токене (смена пароля мгновенно отзывает все токены), проверку роли и DTO-валидацию с `whitelist`. Права на конкретный объект (своя заявка, подчинённый) проверяет сервис. Ошибка любого уровня возвращается как `application/problem+json` с кодом из каталога (документ 2).

6 Клиентская часть

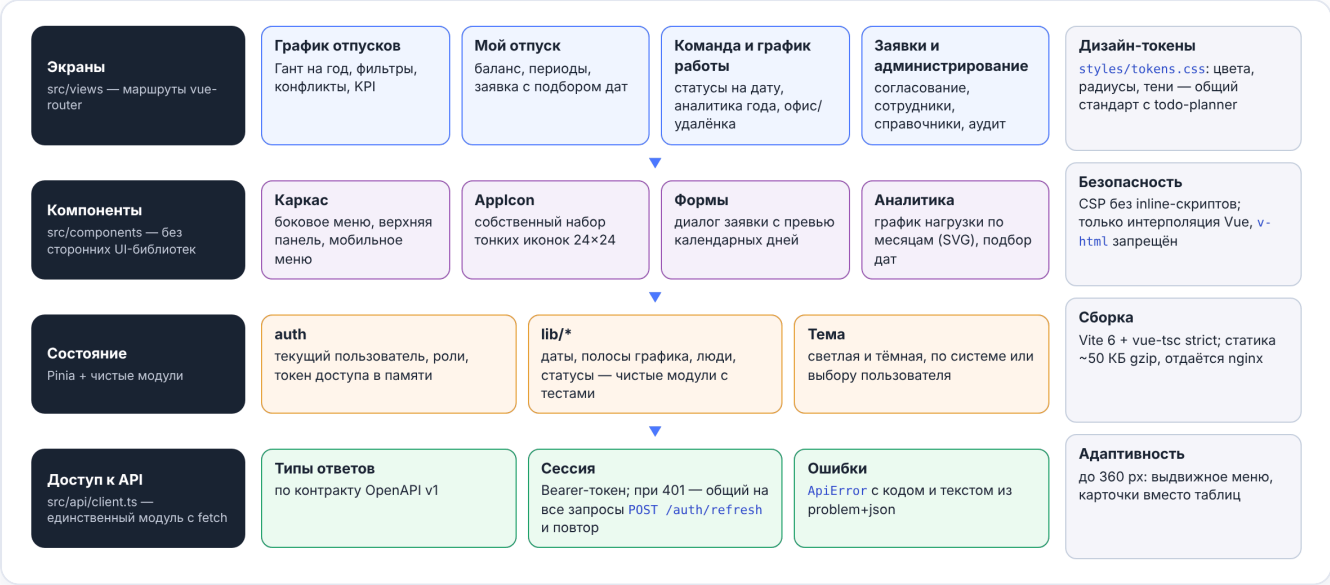


Рис. 7. Слои клиента

Клиент — Vue 3 + TypeScript strict + Vite + Pinia без сторонних UI-библиотек: дизайн-система (токены, иконки, паттерны) общая с внутренними продуктами todo-planner и almi_license. Единственный модуль, обращающийся к сети, — `web/src/api/client.ts`: он добавляет Bearer-токен, при `401` один раз обновляет его по httpOnly-cookie (параллельные запросы ждут общий результат обновления, иначе ротация отозвала бы токен) и превращает problem+json в `ApiError`.

ЭКРАН	НАЗНАЧЕНИЕ
График отпусков	Гант года: масштабы год/квартал/месяц, группы отделов, выходные и праздники, линия «сегодня», полоса нагрузки, слой удалёнки, сравнение выбранных сотрудников, выгрузка в Excel
Мой отпуск	Баланс с разбивкой, ближайший отпуск, год одним взглядом, заявка с подбором дат, перенос и отмена
Команда	Кто сегодня в офисе, на удалёнке, в отпуске; аналитика года: выводы, нагрузка по месяцам, конфликты, распределение дней
График работы	Недельный шаблон офис/удалёнка и разовые исключения на календаре

ЭКРАН	НАЗНАЧЕНИЕ
Заявки	Очередь согласования с пересечениями в отделе, отклонение с причиной, оформление отпуска за подчинённого
Администрирование	Сотрудники, справочники и производственный календарь, правила организации, журнал аудита

Тема оформления — светлая, тёмная или по настройке системы; интерфейс адаптирован до ширины 360 px.

7 Модель данных

Схема описана в `server/prisma/schema.prisma` (14 таблиц PostgreSQL); полный словарь данных и инварианты — документ 4.

7.1 Организация

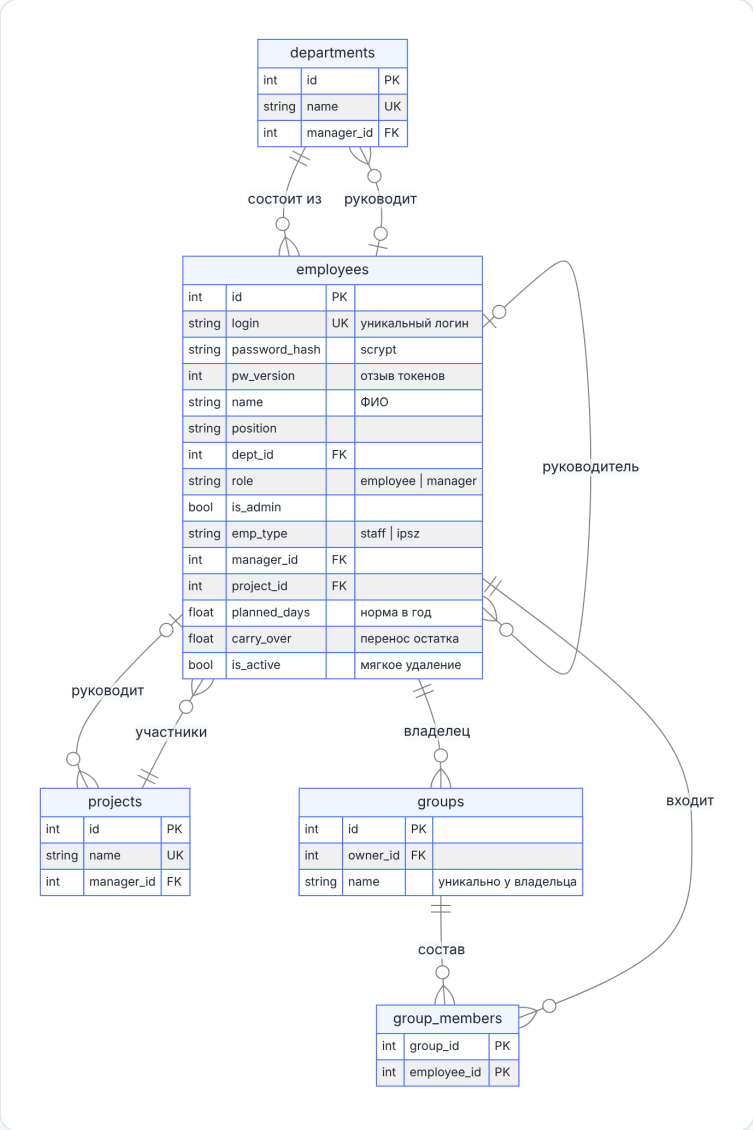


Рис. 8. ER: сотрудники, отделы, проекты, группы

7.2 Отпуска и режим работы

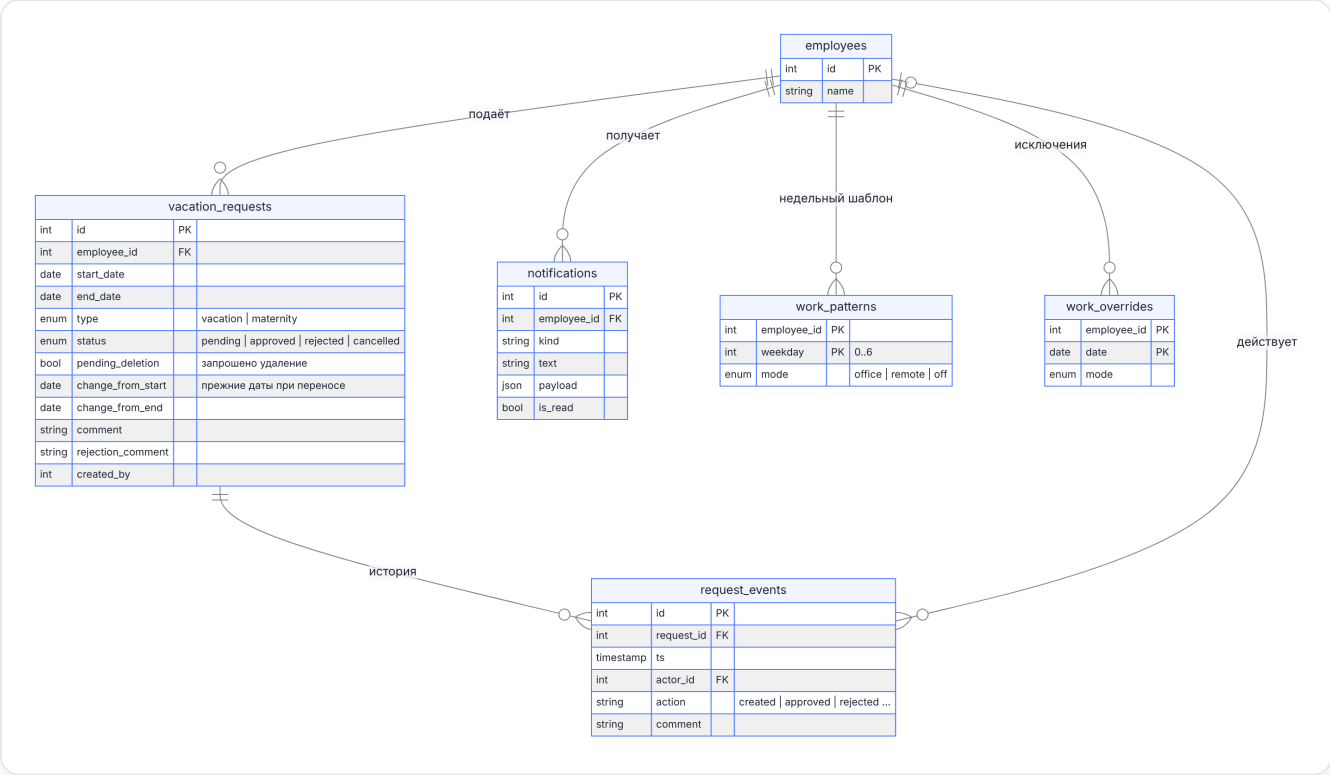


Рис. 9. ER: заявки, история, уведомления, режим работы

7.3 Служебные данные

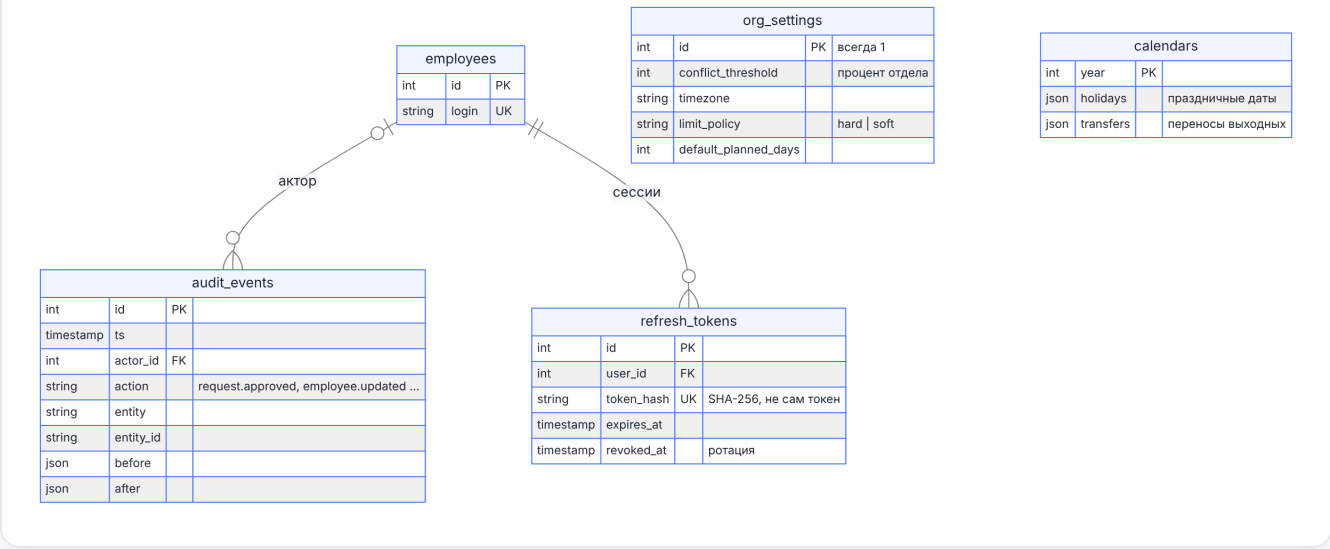


Рис. 10. ER: аудит, сессии, настройки, календарь

7.4 Жизненный цикл заявки

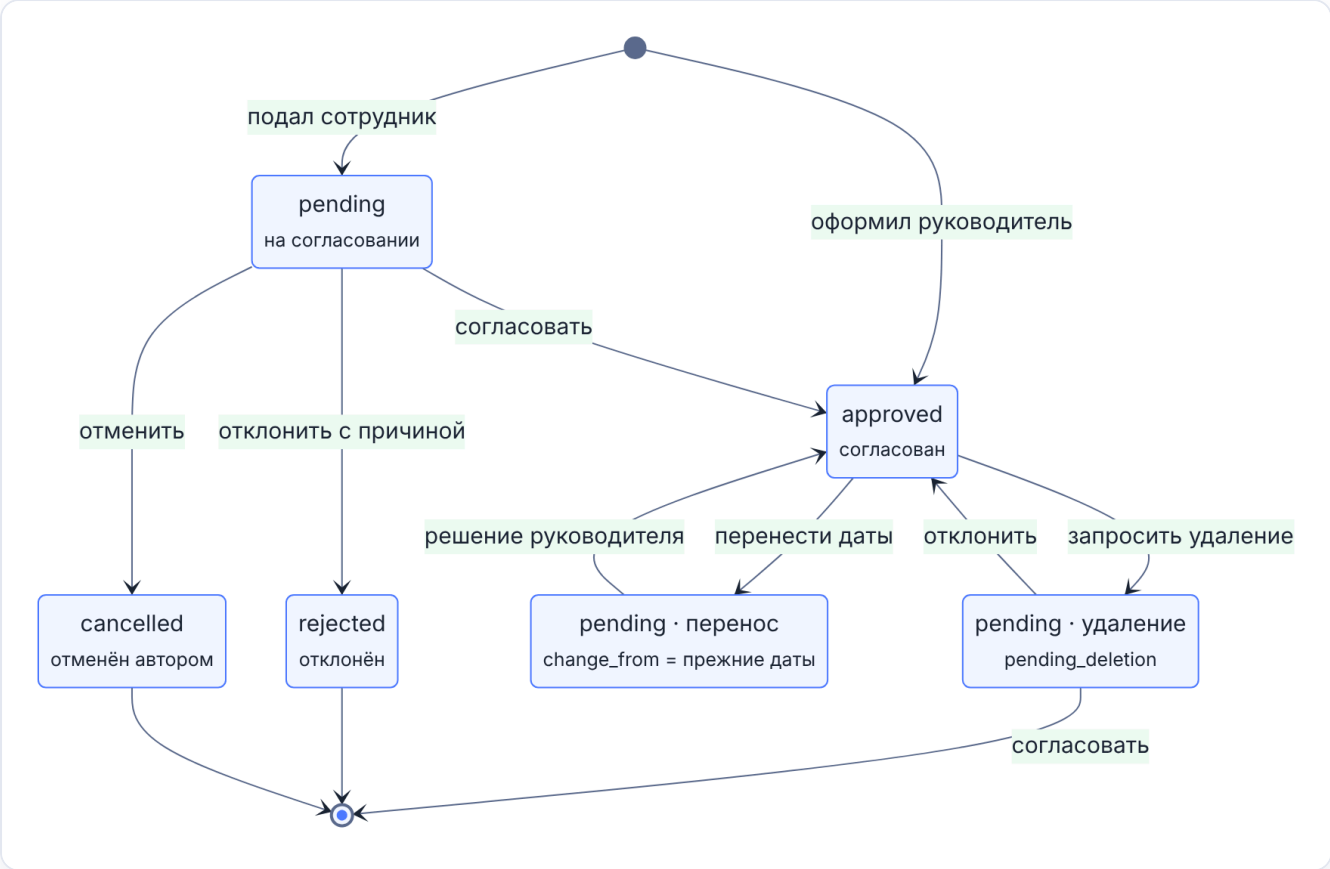


Рис. 11. Состояния заявки

8 Ключевые сценарии

8.1 Вход и продление сессии

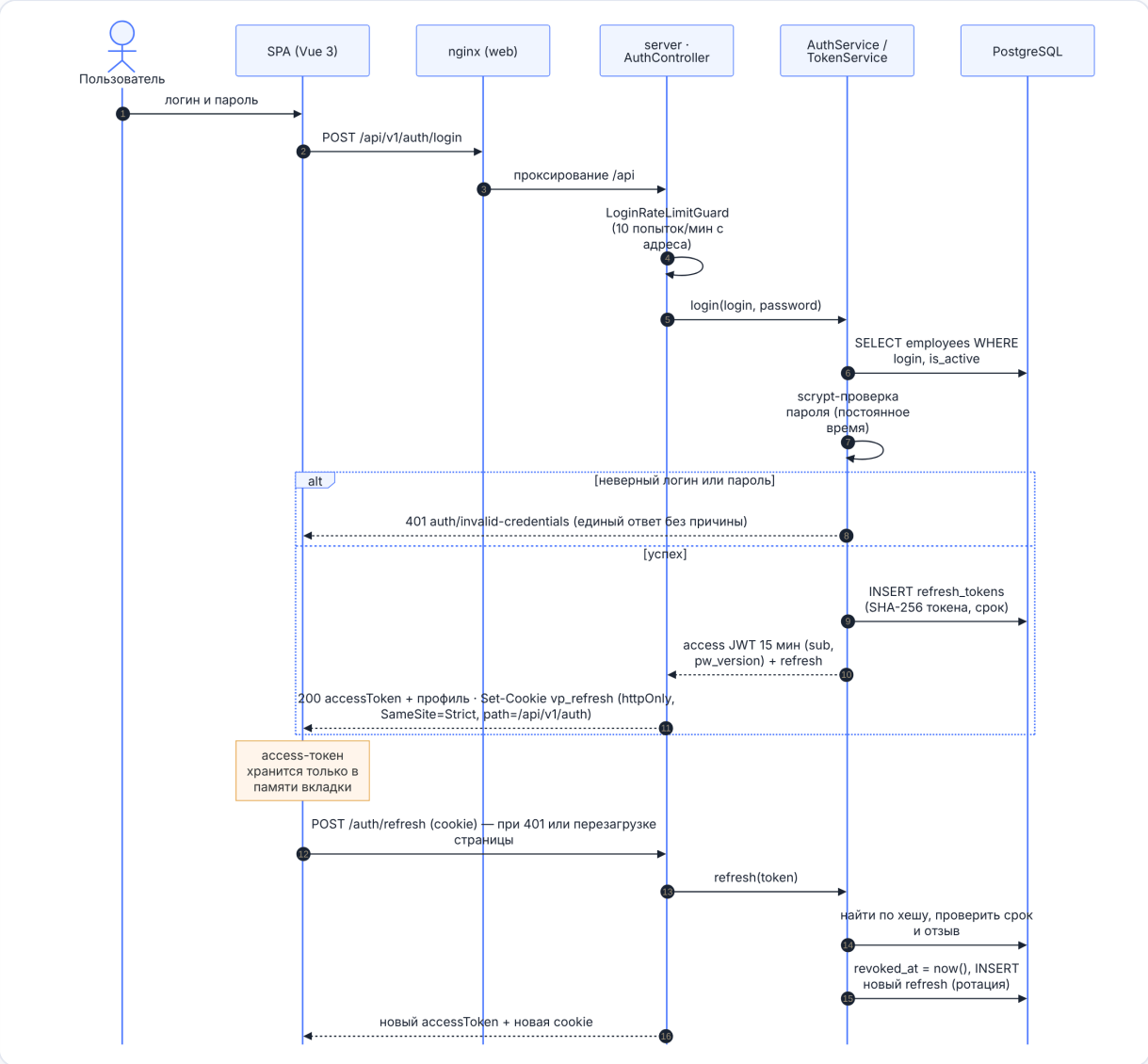


Рис. 12. Последовательность входа и обновления токена

8.2 Подача заявки

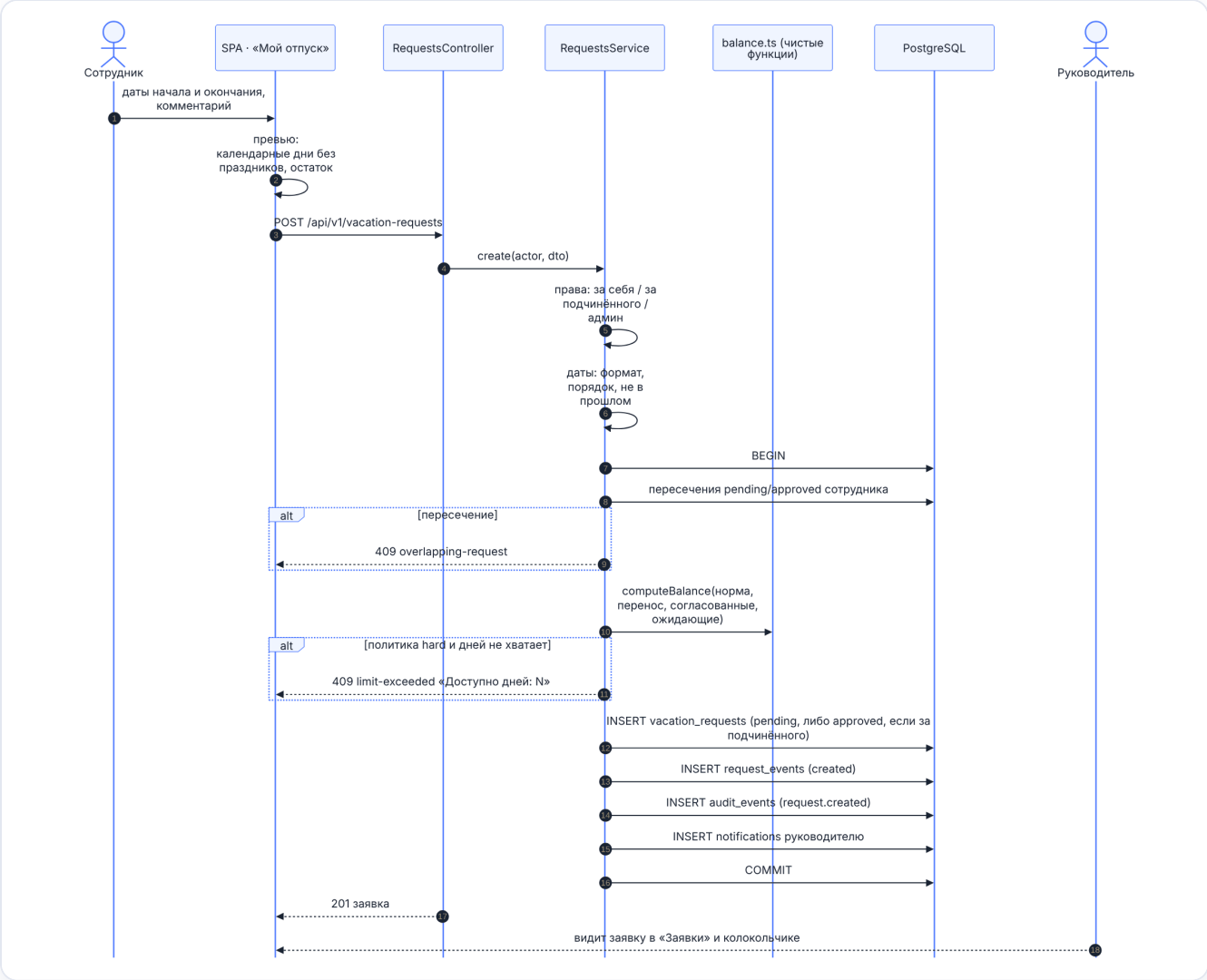


Рис. 13. Создание заявки: проверки и транзакция

8.3 Согласование и отклонение

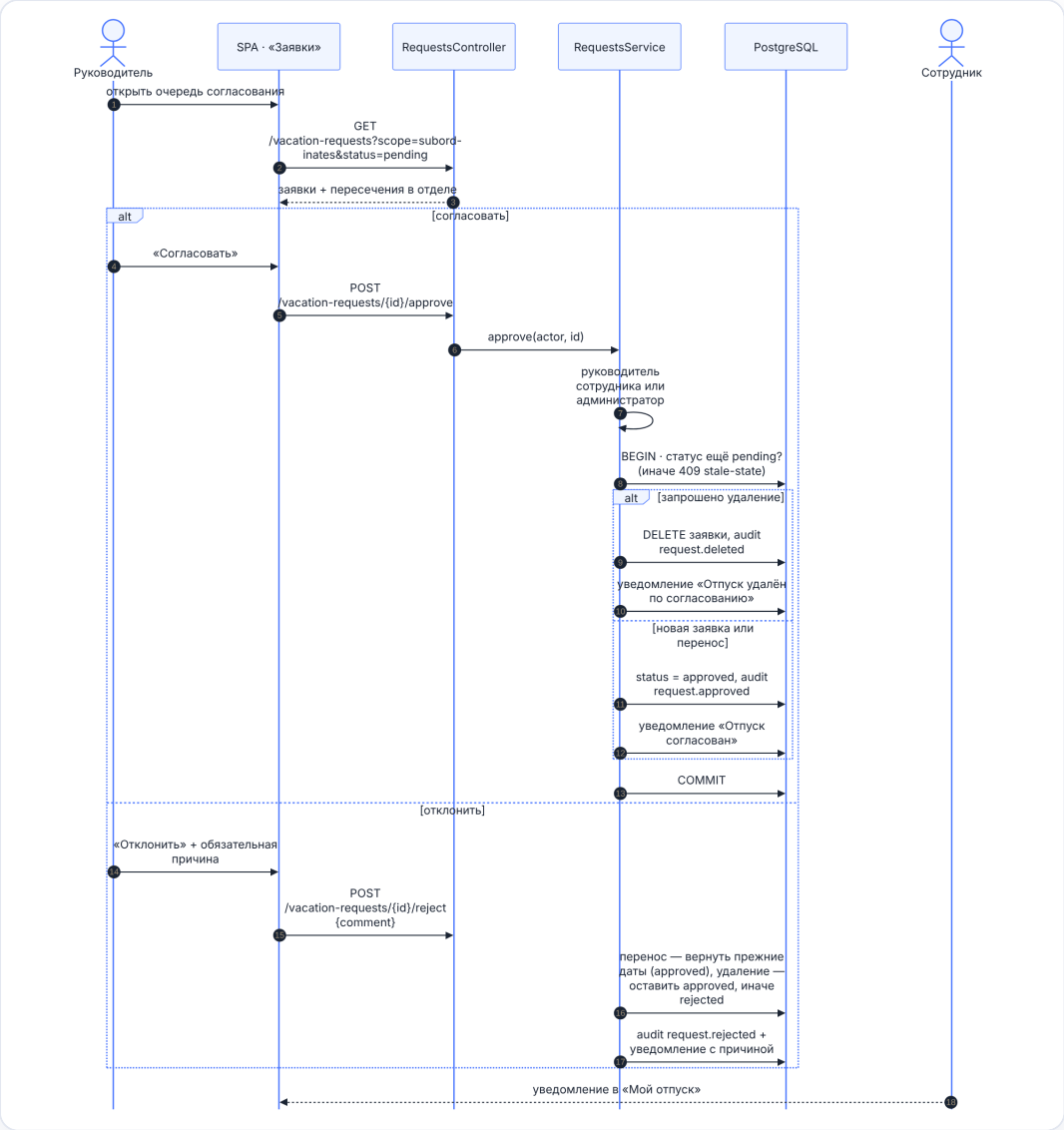


Рис. 14. Решение руководителя

8.4 Построение графика

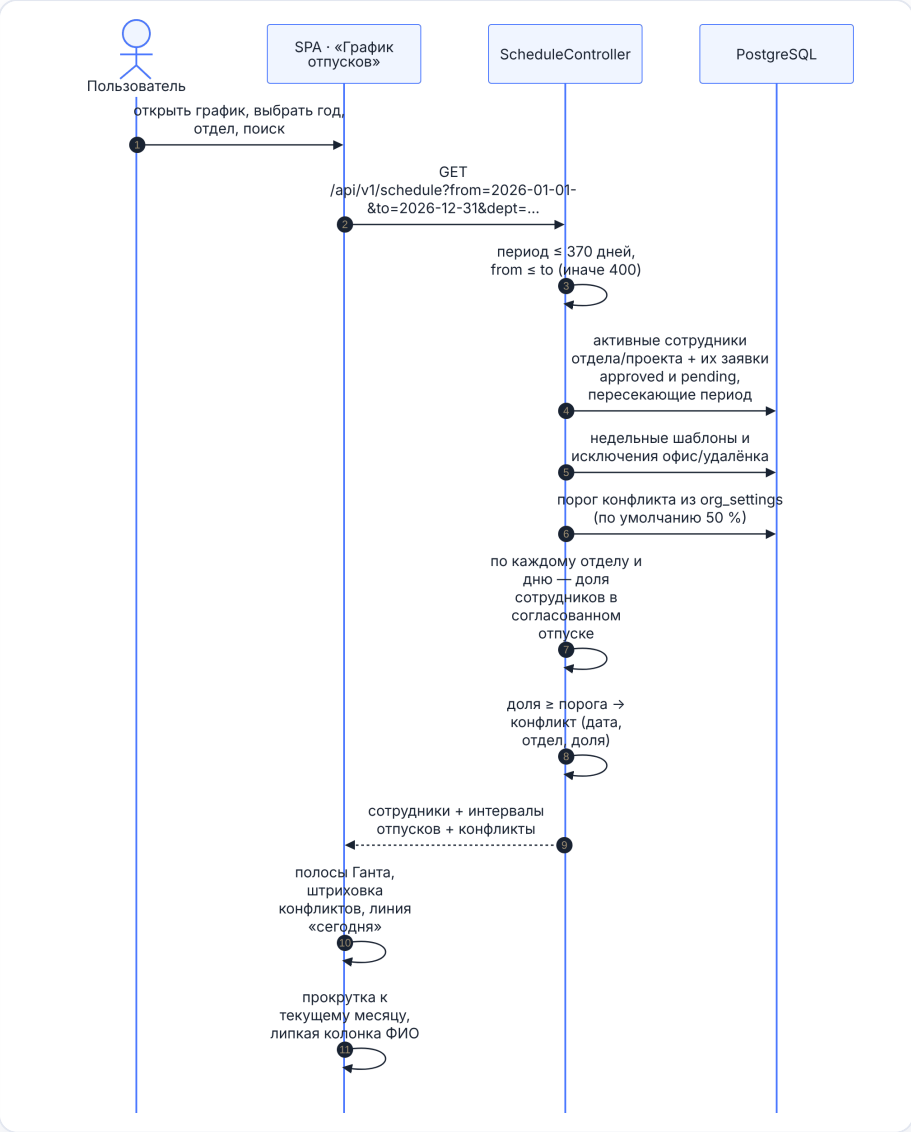


Рис. 15. Загрузка графика с конфликтами отделов

9 Безопасность

9.1 Аутентификация и сессии

МЕХАНИЗМ	РЕАЛИЗАЦИЯ
Хранение паролей	scrypt с солью, сравнение за постоянное время (`auth/passwords.ts`); пароли не попадают в журналы, ответы и выгрузки
Access-токен	JWT HS256, 15 минут, в памяти вкладки (не в localStorage)
Refresh-токен	Случайный, в httpOnly-cookie `vp_refresh` (SameSite=Strict, Secure в production, path `/api/v1/auth`); в БД хранится только SHA-256
Ротация	Каждое обновление отзывает использованный refresh и выдаёт новый
Смена пароля	Увеличивает `pw_version` : все выданные токены перестают действовать
Ограничение частоты	10 попыток в минуту на пару «адрес + логин» и 60 на адрес → `429` с `Retry-After` ; адрес клиента берётся из `X-Forwarded-For` только от доверенных прокси частных сетей (`TRUST_PROXY`)
Ответ на неверный вход	Единый `401 auth/invalid-credentials` без указания причины

9.2 Защита запросов

Глобальные guards применяются ко всем маршрутам по умолчанию; публичные (`/healthz` , `/readyz` , календарь, вход, обновление) отмечены явно. DTO-валидация отбрасывает неизвестные поля. Ответы API содержат `X-Content-Type-Options: nosniff` , `X-Frame-Options: DENY` , `Referrer-Policy` , `Permissions-Policy` . nginx отдаёт SPA с Content Security Policy (`script-src 'self'` , без inline-скриптов, `frame-ancestors 'none'`); e2e-тесты выполняются под той же политикой и падают при её нарушении. В клиенте запрещён `v-html` : данные пользователей выводятся только интерполяцией Vue с автоэкранированием.

9.3 Модель авторизации

РОЛЬ	ПРАВА
Сотрудник	Свои заявки, баланс и график работы; общий график; команда и аналитика своего отдела; подбор дат для себя
Руководитель	Дополнительно: согласование заявок и оформление отпусков прямых подчинённых, их режим работы, аналитика отделов, где работают подчинённые или которыми руководит
Администратор	Всё: сотрудники, справочники, календарь, правила, аудит, выгрузки организации; согласование любых заявок; последнего администратора разжаловать нельзя

9.4 Журналирование и аудит

Каждое изменение данных пишет запись в `audit_events` (кто, когда, действие, объект, значения до и после) в той же транзакции, что и само изменение: запись журнала не может потеряться или появиться без изменения. Журнал только дополняется. Администратор просматривает его с фильтром по объектам и раскрытием изменённых полей.

10 Программный интерфейс

REST API v1 — JSON по HTTPS с префиксом `/api/v1`, описание — OpenAPI 3.1 в `contracts/`. Реализация публикует интерактивную спецификацию на `/api/docs`. Полный каталог операций с правами и кодами ошибок — документ 2.

ГРУППА	ОСНОВНЫЕ ОПЕРАЦИИ
Сессии	вход, обновление, выход, профиль, смена пароля
Заявки	список (свои, подчинённых, все), создание, изменение и перенос, отмена или запрос удаления, согласование, отклонение
График и режим	график за период с конфликтами; шаблон и исключения режима работы
Аналитика	доступные отделы, обзор года, состав на дату, подбор дат
Справочники и настройки	отделы, проекты, группы, календарь, правила организации
Сервис	уведомления, журнал аудита, выгрузка XLSX, пробы

11 Наблюдаемость и эксплуатация

ЗАДАЧА	СРЕДСТВО
Живость и готовность	`GET /healthz` (процесс), `GET /readyz` (запрос к БД); healthcheck контейнера PostgreSQL
Журналы	stdout контейнеров (`docker compose logs`), уровни log/warn/error
Резервное копирование	Сервис `backup` : `pg_dump -Fc` раз в сутки, хранение 14 дней; восстановление — `pg_restore` (руководство по развёртыванию)
Обновление	`git pull` и `docker compose up -d --build` ; миграции применяются при старте `server`
Целевые показатели	RPO 24 ч (ежесуточная копия), RTO 1 ч; ответ p95 ≤ 300 мс на чтение

12 Технологический стек

СЛОЙ	ТЕХНОЛОГИИ
Сервер	Node.js 20/22, NestJS 11, TypeScript 5 strict, class-validator, exceljs
Данные	PostgreSQL 17, Prisma 6 (схема, миграции, клиент)
Клиент	Vue 3.5, Vite 6, Pinia, vue-router 4, собственные компоненты и иконки
Тесты	Vitest + supertest (API на тестовой БД), Vitest (модули клиента), Playwright (e2e интерфейса)
Качество	ESLint 9, vue-tsc, тест сверки контракта, лицензионный гейт SBOM (CycloneDX 1.5)
Поставка	Docker multi-stage, docker compose, nginx; GitHub Actions и Gitea Actions
Документация	Markdown → pandoc → PDF (Chromium) и DOCX; схемы Mermaid и HTML → PNG

13 Ключевые решения

РЕШЕНИЕ	АЛЬТЕРНАТИВА	ОБОСНОВАНИЕ
Модульный монолит NestJS	Микросервисы	Масштаб до 5000 пользователей обслуживается одним процессом; DI, guards и OpenAPI из коробки; модули повторяют предметные области
PostgreSQL с первой версии	SQLite с последующим переходом	Конкурентная запись, транзакции, репликация; нет миграционного долга
Prisma	TypeORM, SQL вручную	Типобезопасные запросы, схема как документация, миграции с ревью SQL
JWT + refresh с ротацией в БД	Серверные сессии	Сервер без состояния для горизонтального масштабирования; отзыв через таблицу
scrypt из `node:crypto`	bcrypt, argon2	Без нативных зависимостей, достаточная стойкость
Свой UI без библиотек	PrimeVue, Quasar	Единая дизайн-система продуктов; минимальный SBOM для реестра
Аудит в транзакции изменения	Асинхронные события	Журнал всегда согласован с данными
Аналитика на сервере чистыми функциями	Расчёт в браузере	Одинаковые правила для всех клиентов; тестируемость; данные других отделов не уходят в браузер

Полные записи — `docs/adr/`.

14 План развития

ВЕРСИЯ	СОДЕРЖАНИЕ	СТАТУС
v0.1.0	Зрелый стек вместо прототипа: вход, заявки, график, справочники, аудит, Docker, SBOM, CI	выпущена
v0.2.x	Схемы документации, XLSX-выгрузки, ограничение частоты входа, security-заголовки, полосы Ганта	выпущена
v0.3.0	Конвейер документов и новые схемы; новый интерфейс и иконки; праздники в балансе; аналитика пересечений и подбор дат; график работы, панель команды, сравнение; резервное копирование; e2e интерфейса и сверка контракта в CI	текущая
v0.4.0	Интеграции: импорт из кадровой системы, единый вход (OIDC/AD), письма о решениях, отдельная роль кадровика	план
v0.5.0	Эксплуатация на 5000: метрики Prometheus, нагрузочная валидация, реплика PostgreSQL (RPO 15 мин), живые обновления	план
v1.0.0	Приёмка HR и ИБ, комплект для реестра отечественного ПО	план